

Question 1:

Steps

1. GET NUMBER OF WRONG LETTERS (different from goal state)
RETURN THAT NUMBER FOR THE HEURISTIC
2. It lower bounds the unavoidable cost of minimum number of steps to change from state to goal. For a state, the most number of steps required to turn it into the goal state depends on the number of letters that are currently wrong for the goal state.
3. The heuristic cannot overestimate the true optimal remaining cost because it calculates the exact minimum cost for a state to reach the goal state, assuming the path tries to change a wrong letter for each iteration.

Scrabble

1. FIND WRONG LETTERS
KEEP TRACK OF THEIR VALUES
2. The heuristic lower bounds the unavoidable cost of switching letters individually. For scrabble, each changed letter has a cost, so finding those wrong letters and adding up their values gives the minimum cost required.
3. The heuristic cannot overestimate the true optimal remaining cost because it calculates the exact minimum cost for a state to reach the goal state, assuming the path tries to minimize the number of switched letters.

Frequency

1. FIND NUMBER OF WRONG LETTERS
ADD THAT NUMBER TO THE RARITY OF THE GOAL
2. The heuristic lower bounds the cost of the rarities of intermediate words. The different words in the goal path will have different rarities, which cannot exactly be calculated. The only guaranteed rarity would be from the goal state. The formula also is $1 + \text{rarity}(w)$, so each wrong word has 1 cost at minimum.
3. The heuristic cannot overestimate the true optimal remaining cost because adding the # of wrong letters and the rarity of the goal word basically ignores the rarity of the intermediate words, which would have added to the true cost.

Question 2:

Word pair	Cost measure	UCS cost	UCS nodes expanded	A* cost	A* nodes expanded	Reduction
WHY → ASK	Steps	5	773	5	16	97.9
WHY → ASK	Scrabble	9	765	9	13	98.3
WHY → ASK	Frequency	38.855	808	38.855	634	21.53
BEAR → DUCK	Steps	4	691	4	6	99.13
BEAR → DUCK	Scrabble	11	2981	11	6	99.79
BEAR → DUCK	Frequency	38.24	1084	38.24	302	72.14

1. Yes, UCS and A* gave the same optimal cost
2. For WHY->ASK, **steps** gave the biggest reduction. For BEAR->DUCK, **scrabble** gave the biggest reduction.
3. For both word pairs, the **frequency** cost measure had the least informative heuristic.
4. With frequency, my heuristic definition used the wrong_counter + rarity(goal). For scrabble, my heuristic calculated the cost of each incorrect letter, and for steps, my heuristic calculated the number of incorrect letters. Scrabble and Steps had heuristics that gave the most accurate value for the cost, but frequency did not include the rarity of the intermediate words between the current state and the goal. This means frequency's heuristic was not as close to the real cost compared to steps and scrabble.

Question 3:

Steps

1. Yes
2. The optimal cost from W1 to W2 depends on the number of mismatched letters. This cost is the same for W2 to W1. For example, DOG to CAT would have a cost of 3, while CAT to DOG would also have a cost of 3.

Scrabble

1. No
2. The optimal cost for scrabble depends on the resulting letter after being changed, which could be different for W1 to W2 and W2 to W1.
3. For "bury" to "coin", the scrabble cost would be 1 + 1 + 1, which is 3. For "coin" to "bury", the cost would be 1 + 1 + 4, which is 6.

Frequency

1. No
2. The optimal cost for frequency depends on the intermediate words between the initial and goal words, which can be different for W1 to W2 and W2 to W1.
3. duck->bear had a cost of 36.79 when using frequency, but bear->duck had a cost of 38.24.

Question 4:

1. ChatGPT 5.6 Sol, "In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary. The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal. By "more informative," we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.",
2. proposed heuristic:

The proposed heuristic

For a state n of length L and goal g (same length), let

$$h_0(n) = \#\{i : n_i \neq g_i\}$$

be the Hamming-distance baseline, and let

$$m_L = \min_{w \in \text{Dict}_L} \text{rarity}(w)$$

be the smallest rarity value among all dictionary words of length L (computed once, offline, in $O(|\text{Dict}_L|)$ time — this is a global constant, not a per-state computation).

Define

$$h_1(n) = \begin{cases} 0 & n = g \\ h_0(n) + (h_0(n) - 1) m_L + \text{rarity}(g) & n \neq g \end{cases}$$

3. The heuristic is admissible since it calculates 0 when all the letters are the same. When the letters differ, the heuristic sums up the rarity of the goal, the number of wrong letters, and the (minimum rarity)*(# of wrong letters - 1). This heuristic gets a good idea of expected cost from the current state to the goal state as it is similar to mine, but includes a minimum rarity.
4. The proposal is valid
5. For WHY->ASK, the heuristic got 38.86 cost and 808 nodes expanded for UCS; and 38.86 and 634 nodes expanded for A*.
For BEAR->DUCK, the heuristic got 38.24 for both costs, with 1084 nodes expanded for UCS and 301 nodes expanded for A*.
These values are pretty much the same as my heuristic, so the Claude heuristic is good.